

RAPPORT DE PROJET

Algorithmique et Structures de Données

Système de Gestion d'un Hôpital D'urgence

Réalisé par :

-Medjedoub Chihab
-Moussaoui Juba
-Belil Rayane
-Larbi Ouchiha

Table des matières

Introduction	3
Outils Utilisée :	3
L'organisation du code	3
Choix Algorithmiques et Structures de Données (include/struture.h)	4
Les Listes Chaînées Simple.....	4
Les patients :	4
Les tickets :	4
Les observations :	5
Les Tableaux statiques	5
Médicaments Et Equipement :	5
Lit :	6
Les Enumération.....	6
Les Algorithme Principaux	6
AddPatient (src/reception.c).....	6
AddTicket (src/reception.c)	6
CallPatient (src/medecin.c)	7
AddObservation (src/observation.c)	7
Sauvegarde Et Chargement (src/fichier.c)	8
Admin.h	8
Gestion générale	8
Médicaments	8
Équipements	8
Main.c	9
Problème Reconnu pendant le projet	9
Sauvgarde :	9
Compteur pour nouveau jour : (verifierNouveauJour())	9
Patient et Observation :	9
Verification fin observation : (verifierFinObservations())	10

Introduction

Ce projet porte sur la gestion d'un hôpital d'urgence, qui a pour but d'automatiser le flux des patients. Nous suivrons les patients de leur entrée dans l'établissement jusqu'à leur sortie. Nous avons essayé de traiter le maximum de cas possible en utilisant les notions vues en cours, en plus d'avoir effectué des recherches complémentaires.

Outils Utilisée :

IDE : [Visual Studio Code](#)

Système de contrôle : [Git](#) pour une meilleure organisation entre les membres du groupe.

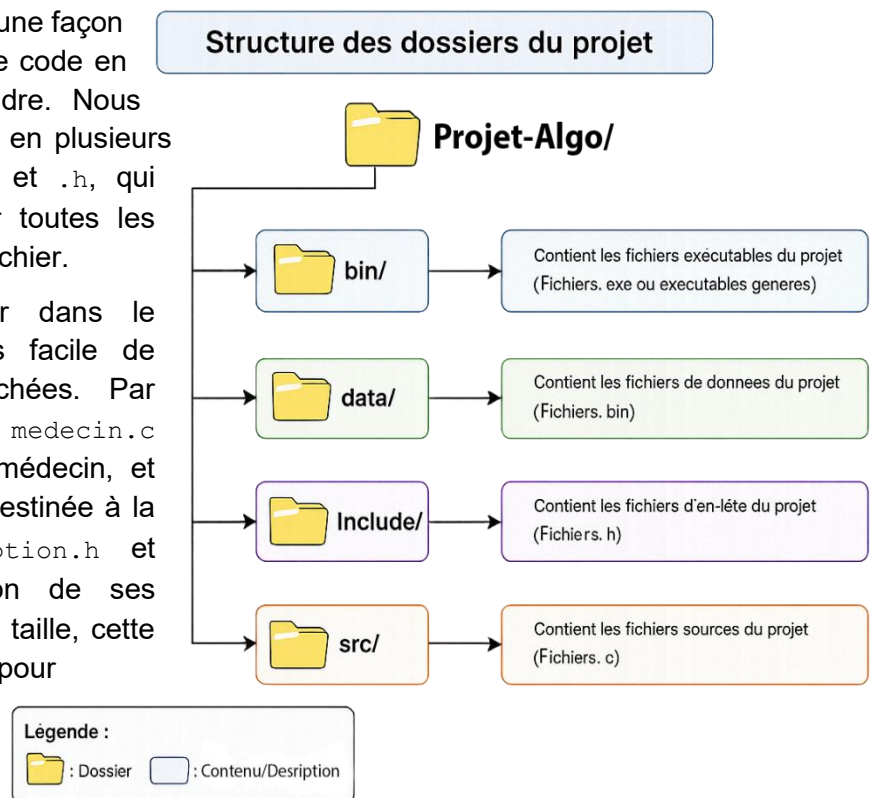
Dépôt de projet : [Github](#) pour la sauvgarde du projet.

Compilation : A un moment, la compilation du projet devient de plus en plus lourde a à cause du grand nombre de fichiers crée, alors on a créé un autre fichier run.bat qui nous permet d'exécuter le projet en 1 seule commande, ./run.bat

L'organisation du code

Nous avons organisé le projet d'une façon intelligente, de sorte que même le code en lui-même soit facile à comprendre. Nous avons notamment divisé le projet en plusieurs fichiers pour les extensions .c et .h, qui permettent de ne pas mélanger toutes les fonctions dans un seul et unique fichier.

Comme on peut le constater dans le diagramme suivant, il est plus facile de retrouver les fonctions recherchées. Par exemple, on retrouve dans src/ medecin.c pour les fonctions destinée au médecin, et reeption.c pour les fonctions destinée à la réception, dans include/ reception.h et medecin.h pour la déclaration de ses fonctions. Pour un projet de cette taille, cette séparation devient essentielle pour une meilleure organisation et ne pas se perdre en plein milieu de programmation.



Choix Algorithmiques et Structures de Données (include/struture.h)

Les Listes Chaînées Simple

Dans un hôpital, plein d'informations nous y sont inconnue, tel que le nombre de patient qui vont arriver, qui vas rester, combien de ticket par jour et on en passe ! Cependant, dans la déclaration d'un tableau dans notre programme, la taille doit être préciser sur le champ. Vu qu'on ne sait pas qu'elle est cette ' taille ', on aura besoins d'une liste chainer qui nous permettra d'incrémenter cette taille aux besoins, et au contraire, la décrémenter, ce qui nous permettra une allocation dynamique pour la mémoire sans las gaspiller. Dans notre cas, on l'aura besoins pour 3 types :

Les patients :

Tel que, un ' Patient ' a beaucoup d'informations, on aura alors besoins d'utiliser les Enregistrement (Structures) pour définir toutes les informations essentielles lors du parcourt du patient jusqu'à la rencontre du médecin.

Mais à l'entrée on aura besoins de mettre ses patients la dans une fille d'attente, pour cela on aura besoins de leur attribuer des tickets, on va alors crée un ' Elément ' ticket dans notre programme, mais combien ? C'est là qu'on aura aussi besoins d'une liste chainer pour ça ! On parlera des [tickets ici](#). Mais d'abord, le type Patient défini qu'un seul patient et pointe vers son suivant, mais dans notre cas, on aura besoins de statistiques de l'hôpital, il nous faudra compteur pour voir combien y a-t-il de patient dans la fille d'attente, on va alors définir un compteur, le souci, c'est qu'on ne va pas demander à un seul Patient dans la file d'attente « combien y a-t-il

```
struct Patient{
    char nom[30];
    char prenom[30];
    int age;
    char id[20];
    char sexe[4];
    Ticket *ticket;
    Etat etat;
    Heure heure;
    int index;
    char diagnostique[100];
    char ordonnance[100];
    char departement[30];
    time_t debutConsultation;
    time_t dureeConsultation;
    struct Patient *suivant;
};
```

```
typedef struct ListePatient {
    Patient *tete;
    int total;
    int attente;
    int observation;
    int sortis;
    int transferes;
}ListePatient;
```

devant toi ? » Car un Patient ne pointe qu'a sont suivant s'il existe, c'est pour ça qu'on aura besoins d'un Elément qui pourra suivre tous les patients et les compter qu'on nommera ListePatient qui contiendra des compteurs pour tout les type dont en aura besoins, et un pointeur vers le premier Patient de la liste chainer.

Les tickets :

Pour organiser le flux vers le médecin, une file d'attente est indispensable, tel que le premier arrivé c'est le premier à sortir, vu qu'on ne sait pas justement combien vont arriver, une Liste chainer est le choix idéal. De même que pour

Patient, les informations essentielles d'un ticket ici n'est que son numéro, mais pour une meilleure organisation et faciliter la recherche dans notre programme, savoir qui le détient est essentiel, on va alors crée une Liste

```
typedef struct Ticket {
    int numero;
    Patient *client;
    struct Ticket *suivant;
}Ticket;
```

```
typedef struct ListeTicket {
    Ticket *tete;
    time_t dernierReset;
    int compteur;
}ListeTicket;
```

chainer avec les 3 champs suivants qu'on va nommer Ticket. Pour la même raison des compteurs, on aura aussi besoins d'un autre Elément qui pourra compter tous les tickets déjà crée, qu'on va alors nommer ListeTicket.

Pour le champ inconnu ici present (dernierReset), on l'expliquera [ici](#)

Les observations :

Après la consultation d'un patient, le médecin a plusieurs choix, soit il prescrit une ordonnance, alors le patient peut disposer, soit il le transfert vers un autre département, ou le mettre en observations, mais pour les mêmes raisons vues précédemment, crée une liste chainer pour cette dernière est le meilleur choix. Les informations essentielles pour un patient en observation, c'est les informations essentielles d'un Patient, le traitement à suivre, la durée de l'observation, les médicaments à prendre et surtout un lit. C'est là qu'on va alors crée une liste chainer Observation, qui contiendra toutes les informations essentielles d'une observation citer auparavant. Pour les meme raison que ListePatient et ListeTicket, on créera un nouveau type qu'on nommera ListeObservation pour pouvoir compter tout les patients mit en observation.

```
typedef struct Observation{
    Patient *patient;
    int lit;
    char traitement[100];
    time_t finObservation;
    time_t debutObservation;
    int index;
    char medicamentsUtilises[10][30];
    int nbMedicaments;
    struct Observation *suivant;
    Medicament *medicaments[10];
    Equipement *equipements[10];
}Observation;
```

Les Tableaux statiques

Contrairement aux patients et aux tickets, on dispose d'informations dont on a déjà le Nombre maximale à atteindre, tel que les lit, les médicaments et les équipements, leur capacité ne change pas et est donner au tout début de l'ouverture de notre programme pour la première fois, c'est pour ça qu'on a alors déclarer des constantes comme seuil pour nos tableau statique, qui sont :

```
#define MAX_LIT 100
#define MAX_MEDICAMENTS 100
#define MAX_EQUIPEMENTS 100
```

Médicaments Et Equipement :

Pour définir un médicament, on aura besoin de son nom et de sa quantité. Mais pour savoir combien de médicament sont ajouter, on a besoins d'un autre compteur, mais ou le mettre ? Impossible dans le tableau en lui-même, du

```
typedef struct Medicament {
    char nom[30];
    int quantite;
} Medicament;
```

```
typedef struct ListeMedicament {
    Medicament medicaments[MAX_MEDICAMENTS];
    int total;
} ListeMedicament;
```

coup, on va suivre la même logique appliquer pour ListeObservation et Les autres « Liste », on suivra la même logique pour les Equipement et ListeEquipement

Lit :

Le nombre de lit est déjà connu dans hôpital, tel que les informations d'un lit sont le numéro, son état et le patient qui l'occupe, on va alors créer un

```
typedef struct ListeLit{  
    Lit Tlit[MAX_LIT];  
    int total;  
    int indispo;  
}ListeLit;
```

nouveau type Lit et sa liste ListeLit.

```
typedef struct Lit{  
    Patient *patient;  
    EtatLit etat;  
    int num;  
}Lit;
```

Les Enumération

Pour ne pas prêter confusions, on utilisera des énumérations pour une meilleure lisibilité du code, bien meilleur que de se souvenir de valeur comme 0 = LIT OCCUPE ou 1 = LIT PAS OCCUPE. Et de même pour les patients, c'est là qu'on comprendra alors l'intérêt des Etat vu dans les structures précédentes

```
typedef enum {  
    ATTENTE,  
    CONSULTATION,  
    OBSERVATION,  
    TRANSFERER,  
    URGENCE,  
    SORTI,  
}Etat;
```

```
typedef enum {  
    OCCUPE,  
    NOCCUPE,  
}EtatLit;
```

Les Algorithmes Principaux

AddPatient (src/reception.c)

Une fonction qui nous permettra d'initialiser et de remplir les champs d'un nouveau Patient (malloc) qu'on va ensuite ajouter à notre liste chaîner ListePatient, pour faire ce dernier, on a 2 cas, si notre liste est vide, alors le début de notre liste est ce nouveau patient, sinon, on parcourt notre liste avec un élément de même type qu'on nommera courant pour arriver jusqu'au dernier avec courant = courant->suivant, pour lier ce dernier au nouveau patient.

```
if(ListeP->tete == NULL) {  
    ListeP->tete = P;  
    ListeP->attente++;  
    ListeP->total++;  
return;  
}  
Patient *courant = ListeP->tete;  
while(courant->suivant != NULL) {  
    courant = courant->suivant;  
}  
courant->suivant = P;  
ListeP->attente++;
```

AddTicket (src/reception.c)

Fonction appelée au même moment de l'enregistrement d'un patient, qui nous permettra de lui attribuer un ticket et le faire entrer dans notre file d'attente, pour cela, on a aussi 2 cas, si notre file d'attente est vide, ce nouveau ticket est le début de ListeTicket, sinon on créera un élément qui nous permettra d'aller jusqu'à la fin et ajouter nouveau ticket, en remplissant au

```
if(ListeT->tete == NULL) {  
    ListeT->tete = nouveauTicket;  
    return nouveauTicket;  
}  
Ticket *courant = ListeT->tete;  
while(courant->suivant != NULL) {  
    courant = courant->suivant;  
}  
courant->suivant = nouveauTicket;  
return nouveauTicket;
```

même moment le champ client de Ticket. En sachant, qu'avant tout cela, on doit d'abord créer le ticket avec la fonction malloc.

CallPatient (src/medecin.c)

Une fonction destinée au médecin, qui lui permettra d'appeler le patient suivant dans la liste, on prend alors le premier patient dans la file d'attente ListeTicket, on change l'état du patient en CONSULTATION, et décrémente le nombre de patient en attente.

Après cela, le médecin pourra commencer la consultation à l'arrivée du patient en question. Mais si un patient est vraiment en urgence absolue ? Il aura pas le temps d'attendre dans la file d'attente, c'est ici qu'on aura besoin alors d'une autre file

```
if(ListeU->tete != NULL) {
    Urgence *urgence = ListeU->tete;
    Patient *patientUrgence = urgence->patient;
    patientUrgence->etat = CONSULTATION;
    ListeU->tete = ListeU->tete->suivant;
    ListeU->attente--;
    patientUrgence->debutConsultation = time(NULL);
    sauvegarderPatients(patientUrgence);
    return patientUrgence;
} else {
    printf("Aucun patient en urgence en file d'attente.\n");
    return NULL;
}
```

```
if(ListeT->tete == NULL) {
    printf("Aucun patient en file d'attente.\n");
    return NULL;
}
Patient *patient = ListeT->tete->client;
patient->etat = CONSULTATION;
patient->debutConsultation = time(NULL);
ListeP->attente--;
sauvegarderPatients(patient);
return patient;
}
```

d'attente mais spécialement pour les patient en urgence absolue qu'on va alors nommer Urgence et ListeUrgence avec la même logique que les autres liste, mais avec comme état URGENCE, ce qui permettra à CallPatient de d'abord vérifier dans ListeUrgence car ils sont plus prioritaires, on va alors l'implémenter comme suis

AddObservation (src/observation.c)

Si le médecin décide de mettre en observations un patient, c'est là qu'on aura besoin de cette fonction. Elle enlève le patient de la file d'attente, elle crée une nouvelle Observation en remplissant ses champs et l'ajoute dans notre ListeObservation, en attribuant un lit automatiquement avec la fonction AddLit

```
ListeP->observation++;
ListeP->attente--;
patientOB->suivant = NULL; // on l'ajoute à
if(ListeO->tete == NULL)
    ListeO->tete = patientOB;
else {
    Observation *courant = ListeO->tete;
    while(courant->suivant != NULL)
        courant = courant->suivant;
    courant->suivant = patientOB;
}
sauvegarderObservations(patientOB);
ListeO->compteur++;
return;
```

```
if(patientEnConsultation->ticket == ListeT->tete) {
    ListeT->tete = ListeT->tete->suivant;
    free(patientEnConsultation->ticket); // On l'enlève de la liste d'attente (debut)
}
else { // milieu et fin
    Ticket *courant = ListeT->tete;
    Ticket *precedent = NULL;
    while(courant != patientEnConsultation->ticket) {
        precedent = courant;
        courant = courant->suivant;
    }
    precedent->suivant = courant->suivant;
    free(courant);
    patientEnConsultation->ticket = NULL;
}
```

qui recherche le premier lit disponible dans notre ListeLit.

En donnant aussi des médicaments depuis ListeMédicament, tel que le patient en observations peut avoir jusqu'à 10 médicaments, le lit est alors mis à OCCUPE et le patient a OBSERVATION. Tout ça ne peut se faire que si le médecin a déjà prescrit un diagnostic.

Sauvegarde Et Chargement (src/fichier.c)

A la fermeture du programme, que sa soit accidentellement ou pas, on perd toutes les informations sur les patients passé et les observations passé, on perd tout depuis notre ram. Pour éviter cela, on va crée des fonctions de sauvegarde pour chaque type de patient et les tickets pour pouvoir recharger ce dernier à chaque ouverture de notre programme.

Pour faire cela, on enregistrera nos informations dans des fichiers binaire qui nous facilitera l'écriture et la lecture de ses informations aux besoins mieux qu'un fichier texte. A chaque exécution d'une instruction qui affecte les informations d'un patient, appelle les fonctions de sauvegarde pour avoir une persistance en temps réel, tel que sauvgarderPatient, sauvgarderObservation et SaveTicket

A chaque ouverture du programme alors, on appellera les fonctions de chargement dans notre programme principale main.c pour récupérer tous les informations passer.

```
chargerLit(&lit);  
chargerTickets(&tickets);  
chargerPatients(&patients,&tickets);  
chargerObservations(&patients,&observations,&lit);  
chargerUrgences(&urgences, &tickets);
```

Admin.h

Gestion générale

GereLit : permet de modifier le nombre de lit présent dans l'hôpital. (qui ne peut pas dépasser MAX_LIT)

AfficherStatistique : affiche tous les compteurs du programme et diferent informations sur l'hôpital.

VerifierFinDeStock : affiche tous les médicaments et équipements en rupture de stock.

Médicaments

AjouterMedicament : Sert à ajouter un nouveau médicament dans le stock après la saisie de son nom, et s'il existe déjà on ajoute la nouvelle quantité à celle déjà existante.

AfficherStock : affiche tous les médicaments disponibles et leur quantité en parcourant le tableau dans ListeMedicament.

UtiliserMedicament : on saisit le nom du médicament à utiliser et on décrémente sa valeur de quantité de 1.

FichierMedicament : stocke tous les médicaments et leur quantité dans un fichier binaire. Cette fonction est appelée à chaque fin des sous-programmes concernant les médicaments, afin de réécrire le fichier à chaque modification.

RafrachirMedicament : utilisée à chaque lancement du programme pour intégrer tous les médicaments présents dans le fichier (sauvegardés lors des exécutions précédentes) à la nouvelle exécution, afin qu'ils ne soient pas perdus.

Équipements

Utilise la même logique que les médicaments, en remplaçant simplement « médicament » par « équipement » dans les noms des sous-programmes

Main.c

Nous voici au fichier principal, celui qui regroupe toutes les fonctions citées dans toutes ces bibliothèques précédentes et les inclut toutes pour que l'exécution puisse se dérouler correctement.

Tout d'abord, on inclut toutes les autres bibliothèques dont on a besoin (celles intégrées au langage et celles que nous avons créées nous-mêmes) pour pouvoir utiliser leurs fonctions dans `main.c`. Puis, nous déclarons toutes les variables dont on a besoin (`ListePatient`, etc.) et on les initialise à `NULL`. Grâce aux fonctions `rafraichir` et `charger`, on récupère les données des exécutions précédentes, avec `verifierNouveauJour` pour un les tickets.

Puis nous arrivons à la boucle `do while` principale qui permet de continuer le programme tant que l'on ne décide pas de le quitter (`choix=4`). On affiche le premier menu (menu principal) et on demande à l'utilisateur de faire un choix. À l'intérieur, il y a une instruction `switch` pour exécuter les instructions correspondant au choix précédent, notamment afficher le menu médecin si `choix=1`, le menu réception pour `choix=2`, et l'administration pour `choix=3`.

Il y a évidemment d'autres instructions qui suivent, ainsi que d'autres boucles imbriquées qui demandent d'autres choix. C'est ici que nous utilisons la majorité des fonctions des bibliothèques que nous avons créées. On peut également revenir au menu précédent en choisissant l'option quitter de chaque sous-menu. Enfin, quand on choisit l'option 4 dans le menu principal, on quitte officiellement le programme en ayant au préalable sauvegardé tous les patients et toutes les structures utilisées dans des fichiers, afin de pouvoir les réutiliser lors de la prochaine exécution du programme.

Problème Reconnu pendant le projet

Sauvgarde :

On ne peut pas sauvegarder des pointeurs dans un fichier binaire, comme solution, on a alors créé des variables dans `fichier.c` contenant les valeurs directes des pointeurs pour pouvoir les stocker dans les fichiers binaires.

Compteur pour nouveau jour : (`verifierNouveauJour()`)

Si un nouveau jour arrive, le compteur de ticket poursuit depuis la première fois de son démarrage, alors on pourra avoir un ticket 300 pour le premier patient d'une journée, c'est pour ça qu'on va renormaliser le compteur à chaque minuit et stocker l'heure de cette réinitialisation dans `ListeTicket` qu'on va ensuite stocker dans un fichier binaire pour pouvoir la repérer au prochain démarrage.

Patient et Observation :

On pouvait bien mettre les champs traitement, lit.... Dans patient, mais cela revient à gaspiller de l'espace mémoire si le patient en question ne va pas

aller jusqu'à l'observation, c'est pour ça qu'on a alors séparer ses champs la de Patient

Verification fin observation : (`verifierFinObservations()`)

Une durée d'observation est donnée, faut alors l'appliquer, quand la date donnée par le médecin est arrivée, on met l'état du patient a `SORTI`